

VOLUME 3

VERIQTA

aws

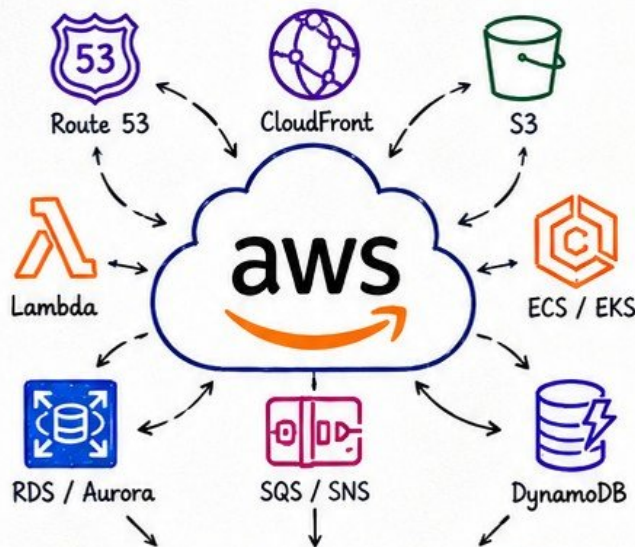
AWS PRODUCTION INCIDENTS

ENTERPRISE ARCHITECTURE, SECURITY,
GOVERNANCE & LARGE-SCALE OPERATIONS

REAL INCIDENTS. REAL IMPACT. REAL SOLUTIONS.

WHAT'S INSIDE?

- ✓ 20 Advanced AWS Production Incidents
- ✓ Enterprise Architecture & Networking
- ✓ Security, Compliance & Governance
- ✓ Hybrid Cloud & Migrations
- ✓ Large-Scale Operations & Optimizations
- ✓ Senior Interview & System Design Prep



YOU WILL LEARN:

- 🔍 Deep Dive Troubleshooting of Advanced AWS Services
- ⚙️ How Senior Engineers Investigate & Resolve Complex Incidents
- 📈 Design for Scale, Resiliency & High Availability
- 🔒 Strengthen Security, Governance & Observability
- ⚡ Prepare for Disasters & Recover with Confidence

MULTI-ACCOUNT STRATEGY



SECURITY ACCOUNT



NETWORK ACCOUNT



SHARED SERVICES ACCOUNT



LOGGING & MONITORING



COMPLIANCE ACCOUNT



AWS ORGANIZATIONS



PRODUCTION
FOCUSED



SENIOR ENGINEER
INSIGHTS



ENTERPRISE
READY



BUILT FOR SCALE
& RESILIENCY

LEARN. TROUBLESHOOT. RECOVER. IMPROVE.
BE A PROBLEM SOLVER. BE VALUABLE.





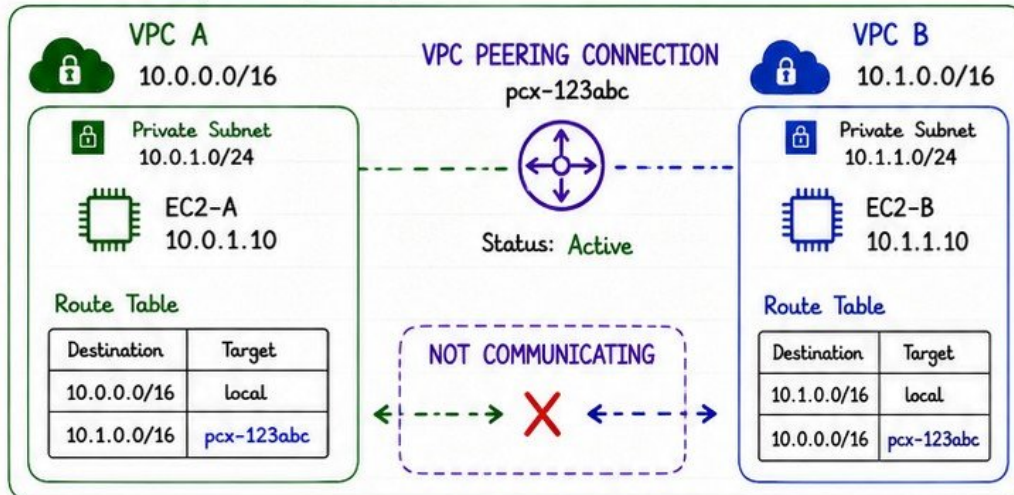
VPC PEERING COMMUNICATION FAILURE



AT A GLANCE

- ✓ Route propagation
- ✓ CIDR overlap
- ✓ Security Groups
- ✓ Network troubleshooting

Applications in two VPCs cannot communicate even though the peering connection is "Active".



WHAT IS VPC PEERING?

VPC Peering enables private connectivity between two VPCs using private IP addresses.

KEY FACT

Peering connection status "Active" only means the connection exists, not that traffic can flow.

COMMON CAUSES



ROUTE PROPAGATION

Routes are not propagated or missing in route tables.



CIDR OVERLAP

Overlapping CIDR blocks between VPCs prevents communication.



SECURITY GROUPS

SG inbound rules do not allow traffic from the peer VPC CIDR.



NACL / OTHER

NACLs, Network ACLs or other network controls blocking traffic.

INVESTIGATION CHECKLIST

- ☐ Verify peering connection status
- ☐ Check VPC CIDR ranges
- ☐ Check route tables in both VPCs
- ☐ Verify route propagation settings
- ☐ Check Security Group inbound rules
- ☐ Test connectivity (ping / telnet)
- ☐ Check NACLs
- ☐ Review Flow Logs / VPC Reachability



Use VPC Reachability Analyzer to quickly identify network path issues.

TROUBLESHOOTING STEPS

- 1 Check Peering Connection Status
→ `aws ec2 describe-vpc-peering-connections \`
→ `--vpc-peering-connection-ids pcx-123abc`
- 2 Verify Route Tables
→ `aws ec2 describe-route-tables \`
→ `--filters Name=vpc-id,Values=vpc-aaa`
- 3 Check Propagation Settings
→ `aws ec2 describe-vpc-peering-connections \`
→ `--vpc-peering-connection-ids pcx-123abc \`
→ `--query 'VpcPeeringConnections[0].Options'`
- 4 Test Connectivity
→ `ping 10.1.1.10 # from EC2-A`
→ `telnet 10.1.1.10 80`

RESOLUTION SUMMARY

- ✓ Ensure CIDR ranges do not overlap.
- ✓ Enable route propagation OR add static routes manually.
- ✓ Update Security Groups to allow traffic from peer VPC CIDR.
- ✓ Verify NACLs are not blocking.
- ✓ Test again and confirm traffic flow.

CORRECT COMMUNICATION FLOW

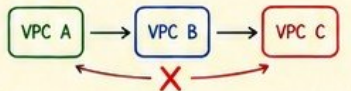


BEST PRACTICES

- ✓ Use non-overlapping CIDR ranges.
- ✓ Enable route propagation for simplicity.
- ✓ Use SG references where possible.
- ✓ Monitor with Flow Logs & Reachability Analyzer.

NOTE

VPC Peering is not transitive. VPC A cannot reach VPC C through VPC B.



TOOLS

- ☐ VPC Reachability Analyzer
- ☐ Flow Logs
- ☐ CloudWatch Metrics
- ☐ Route Tables
- ☐ Security Group References



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA



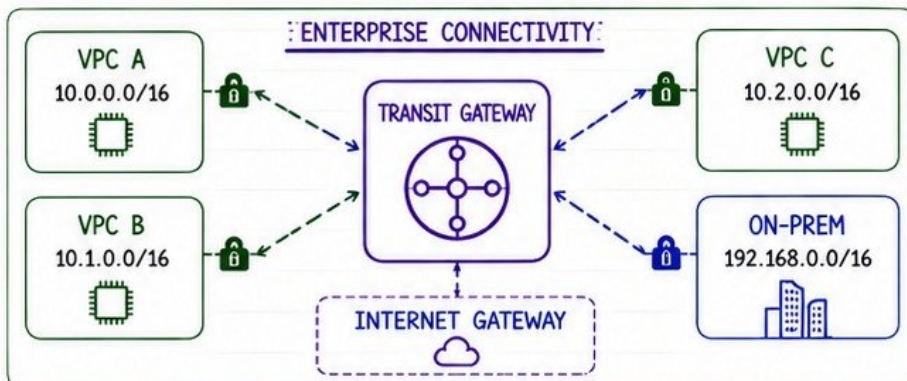
TRANSIT GATEWAY ROUTING FAILURE

Traffic between VPCs and on-premises networks stopped flowing due to incorrect or missing Transit Gateway routing configuration.



AT A GLANCE

- ✓ Route tables misconfigured
- ✓ Missing route propagation
- ✓ Attachment not associated
- ✓ Blackhole or wrong route
- ✓ Enterprise networking impact



WHAT IS AWS TRANSIT GATEWAY?

AWS Transit Gateway (TGW) is a hub that connects multiple VPCs and on-premises networks through a centralized routing hub.

It simplifies network management at scale for enterprise environments.

COMMON CAUSES

- ✗ Routes not propagated to the TGW route table
- ✗ Attachment not associated with route table
- ✗ Blackhole route or incorrect destination
- ✗ Overlapping CIDR blocks
- ✗ Asymmetric routing
- ✗ BGP route not received from DX/VPN
- ✗ Default route missing for on-prem traffic

TRANSIT GATEWAY ROUTE TABLES

Route Table	Associated Attachments	Propagated Routes
Prod-RT	VPC A, VPC B, VPC C	10.0.0.0/16, 10.1.0.0/16, 192.168.0.0/16
Shared-RT	Shared Services VPC	172.16.0.0/12
OnPrem-RT	DX-GW, VPN	192.168.0.0/16, 10.0.0.0/16, 10.1.0.0/16

ATTACHMENTS

- VPC Attachments**
 - Connect VPCs to TGW
- VPN Attachments**
 - Connect via Site-to-Site VPN
- DX Gateway Attachment**
 - Connect via Direct Connect
- Connect Attachments**
 - Connect via AWS Connect

Each attachment must be associated with a route table.

TROUBLESHOOTING STEPS

- 1 Verify TGW Route Tables → `aws ec2 describe-transit-gateway-route-tables`
- 2 Check Route Propagation → `aws ec2 get-transit-gateway-route-table-propagations --transit-gateway-route-table-id tgw-rtb-xxx`
- 3 Check Attachments → `aws ec2 describe-transit-gateway-attachments`
- 4 Check Routes → `aws ec2 describe-transit-gateway-routes --transit-gateway-route-table-id tgw-rtb-xxx`
- 5 Verify BGP (DX/VPN) → `aws ec2 describe-transit-gateway-attachments --filters Name=attachment-type,Values=vpn,direct-connect`
- 6 Test Connectivity → `ping 10.2.0.10 traceroute 10.2.0.10`
- 7 Analyze with Reachability Analyzer → `aws ec2 start-network-insights-analysis --source ... --destination ...`

RESOLUTION SUMMARY

- ✓ Associate attachments to correct route table.
- ✓ Enable route propagation where needed.
- ✓ Remove incorrect or blackhole routes.
- ✓ Fix overlapping CIDRs (if any).
- ✓ Verify BGP & route advertisements.
- ✓ Test and confirm traffic flow.

BEST PRACTICES

- ✓ Use separate route tables per network domain.
- ✓ Enable route propagation for dynamic environments.
- ✓ Use summarization to reduce route table size.
- ✓ Avoid overlapping CIDR blocks.
- ✓ Monitor TGW metrics and route counts.
- ✓ Use Reachability Analyzer for validation.



NOTE

Incorrect routing in TGW can cause complete communication outage across VPCs and on-premise networks.

ROUTES + ASSOCIATIONS = TRAFFIC FLOW

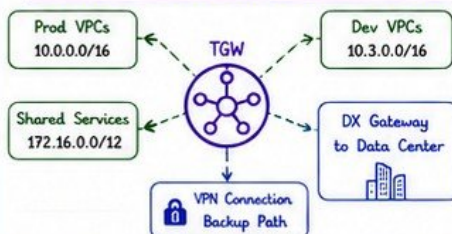
INVESTIGATION CHECKLIST

- ☐ Check TGW route tables
- ☐ Verify route propagation status
- ☐ Check attachment associations
- ☐ Verify routes in each route table
- ☐ Check BGP status (DX/VPN)
- ☐ Test connectivity (ping / traceroute)
- ☐ Review Flow Logs / Reachability Analyzer
- ☐ Check for blackhole routes



Use VPC Reachability Analyzer to validate the network path.

ENTERPRISE TGW ARCHITECTURE (EXAMPLE)



USEFUL TOOLS

- ☐ VPC Reachability Analyzer
- ☐ CloudWatch (TGW Metrics)
- ☐ Flow Logs
- ☐ Route Tables
- ☐ AWS Network Manager



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA



DIRECT CONNECT LINK FAILURE

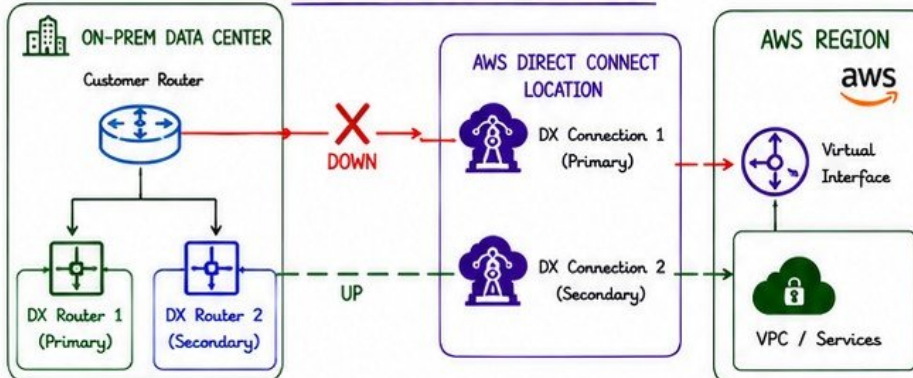


AT A GLANCE

- ✓ Redundancy is critical
- ✓ BGP stability is key
- ✓ VPN failover saves you
- ✓ Quick recovery restores business continuity

On-premises connectivity to AWS lost due to Direct Connect link failure, impacting critical workloads and services.

HYBRID CONNECTIVITY OVERVIEW



WHAT IS AWS DIRECT CONNECT?

AWS Direct Connect provides a dedicated network connection from your data center to AWS.

Benefits:

- Consistent network performance
- Lower latency
- Higher bandwidth
- More secure than internet

INVESTIGATION CHECKLIST

- ☐ Check Direct Connect connection status
- ☐ Verify BGP session status (Established?)
- ☐ Check BGP route table on customer router
- ☐ Check virtual interface status
- ☐ Verify route advertisements
- ☐ Check CloudWatch metrics & alarms
- ☐ Review Direct Connect event history
- ☐ Verify redundancy & HA configuration



Use Reachability Analyzer to test connectivity from VPC to on-prem.

COMMON CAUSES



PHYSICAL LINK FAILURE

Fiber cut, port down, optical signal loss, or provider issue.



BGP SESSION DOWN

BGP session established but route exchange is failing.



ROUTE ADVERTISEMENT ISSUES

Prefixed not advertised due to configuration or policy.



AUTHORIZATION FAILURE

DX connection or virtual interface not authorized.



MAINTENANCE OR PROVIDER ISSUE

Planned or unplanned maintenance at DC or provider side.

TROUBLESHOOTING STEPS

- 1 Check DX Connection Status → `aws directconnect describe-connections`
- 2 Check BGP Session → `show bgp summary`
- 3 Check BGP Routes → `show bgp ipv4 unicast`
- 4 Check Virtual Interface → `aws directconnect describe-virtual-interfaces`
- 5 Check CloudWatch Metrics → `AWS/DirectConnect metrics`
- 6 Verify Route Propagation → `show ip route | include <prefix>`
- 7 Test Connectivity → `ping <VPC endpoint IP>`
- 8 Check Redundant Path → `traceroute <VPC endpoint IP>`

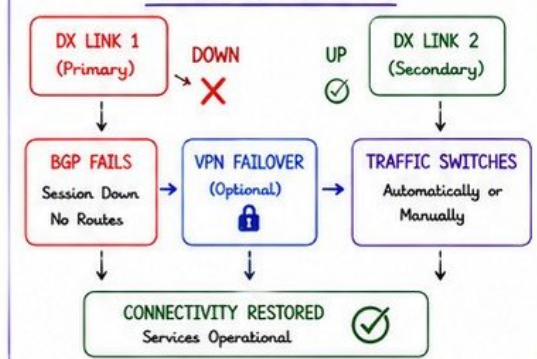
RESOLUTION SUMMARY

- ✓ Restore failed physical link or escalate to provider.
- ✓ Stabilize BGP session and route exchange.
- ✓ Enable correct route advertisements.
- ✓ Ensure redundant DX connection is healthy.
- ✓ Failover traffic to alternate DX link or VPN.
- ✓ Validate end-to-end connectivity.



Goal: Restore stable, redundant connectivity.

FAILOVER & RECOVERY FLOW



BEST PRACTICES

- ✓ Always have dual DX connections in different locations.
- ✓ Use BGP for dynamic routing and faster failover.
- ✓ Monitor Direct Connect metrics and set alarms.
- ✓ Test failover regularly.

KEY METRICS (CLOUDWATCH)

- ConnectionState
- BgpStatus
- BgpSessionState
- InboundBytes / OutboundBytes
- PacketsDropCount



USEFUL TOOLS

- Reachability Analyzer
- VPC Flow Logs
- CloudWatch Dashboards
- AWS Health Dashboard
- Direct Connect Test Connectivity



NOTE

Redundancy + BGP stability + Monitoring = Resilient Hybrid Connectivity



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA



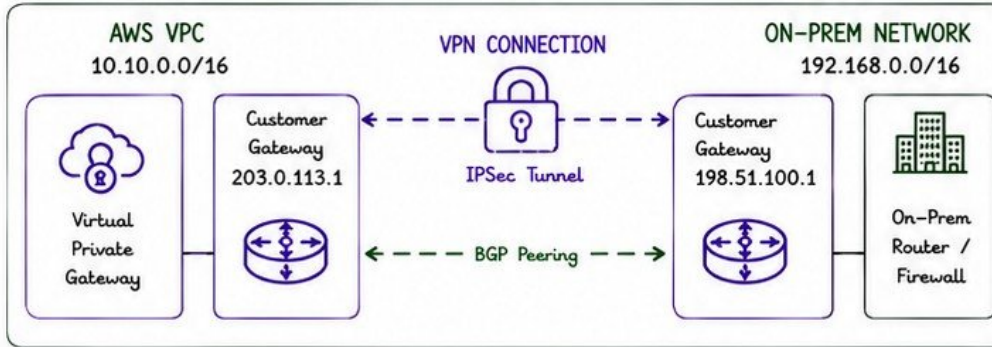
SITE-TO-SITE VPN TUNNEL DOWN

TUNNEL STATUS · IPSEC · BGP · FAILOVER

AT A GLANCE

- ✓ Check tunnel status
- ✓ Verify IPsec
- ✓ Validate BGP
- ✓ Execute failover workflow
- ✓ Restore connectivity fast

A site-to-site VPN tunnel failure breaks connectivity between your on-premises network and AWS, affecting applications, users and critical services.



WHAT IS A SITE-TO-SITE VPN?

A Site-to-Site VPN creates an encrypted tunnel between your on-premises network and AWS over the internet using IPsec.

Components:

- Virtual Private Gateway (VGW)
- Customer Gateway (CGW)
- VPN Connection
- IPsec Tunnels
- BGP (dynamic routing)

TUNNEL STATUS CHECK

AWS Console Path:
VPC Dashboard > Site-to-Site VPN
Connections > Select VPN

Key Status Indicators:

- ✓ Tunnel State: UP
(Both tunnels should be UP)
- ✗ Tunnel State: DOWN
(One or both tunnels DOWN)

💡 You have 2 tunnels per VPN connection for HA.



IPSEC CHECKLIST

- ☐ Phase 1 (IKE) is established
- ☐ Phase 2 (IPsec SA) is established
- ☐ No authentication failures
- ☐ Encryption & integrity match
- ☐ Lifetime not expiring constantly
- ☐ DPD (Dead Peer Detection) enabled
- ☐ Pre-shared keys match



BGP CHECKLIST

- ☐ BGP session is Established
- ☐ Correct ASN on both sides
- ☐ Correct BGP inside IP addresses
- ☐ Routes are being exchanged
- ☐ No route flap or resets
- ☐ Check BGP hold time & timers
- ☐ Verify prefix advertisements

COMMON CAUSES



- ✗ Internet connectivity loss
- ✗ IPsec configuration mismatch
- ✗ Expired certificates / PSK mismatch
- ✗ BGP session down
- ✗ Routing changes / missing routes
- ✗ Firewall blocking UDP 500 / 4500
- ✗ MTU / Fragmentation issues
- ✗ ISP or device failure

TROUBLESHOOTING STEPS

- 1 Check VPN Tunnel Status → `aws ec2 describe-vpn-connections`
- 2 Check Tunnel Details → `aws ec2 describe-vpn-connections --vpn-connection-ids vpn-xxxx`
- 3 Check IPsec Status → `aws ec2 describe-vpn-connections --include-tunnel-information`
- 4 Check BGP Status → `aws ec2 describe-vpn-connections --include-tunnel-information`
- 5 Check BGP Routes → `aws ec2 describe-vpn-connections --include-tunnel-information`
- 6 Verify Customer Gateway → `aws ec2 describe-customer-gateways`
- 7 Check CloudWatch Metrics → `VPNTunnelState, TunnelDataIn, TunnelDataOut`
- 8 Test Connectivity → `ping / traceroute / pathping`

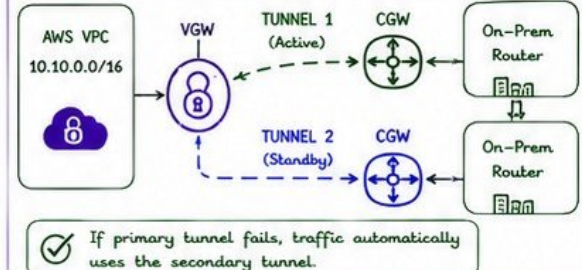


Use Reachability Analyzer to validate network path after recovery.

FAILOVER WORKFLOW

- 1 Detect Tunnel Down
Alert / Monitoring / User Report
- 2 Validate Impact
Confirm affected routes & services
- 3 Check Secondary Tunnel
If one tunnel is DOWN, other tunnel should remain UP
- 4 Verify BGP Route Propagation
Ensure routes are still reachable
- 5 If Both Tunnels DOWN
Trigger VPN Failover / Backup Path
- 6 Validate Connectivity
Test routes, apps and users
- 7 Monitor & Stabilize
Watch metrics and logs

VPN HIGH AVAILABILITY ARCHITECTURE



REQUIRED VPN PORTS

Protocol	Port	Direction	Purpose
UDP	500	Inbound/Outbound	IPsec IKE (Phase 1)
UDP	4500	Inbound/Outbound	IPsec NAT-T (Phase 2)

IMPORTANT NOTES

- ★ Both tunnels must be UP for full HA.
- ★ BGP keeps routes updated during failover.
- ★ Keep configurations consistent on both sides.
- ★ Monitor VPN metrics & configure alerts.



BEST PRACTICES

- ✓ Enable BGP for dynamic routing.
- ✓ Use two tunnels for HA.
- ✓ Keep configurations consistent.
- ✓ Monitor tunnel health continuously.
- ✓ Test failover regularly.
- ✓ Document backup and recovery steps.



KEY CLOUDWATCH METRICS

- VPNTunnelState (Up/Down)
- TunnelDataIn
- TunnelDataOut
- TunnelPacketsIn
- TunnelPacketsOut
- TunnelConnectionErrors



USEFUL TOOLS

- AWS Reachability Analyzer
- VPC Flow Logs
- CloudWatch Dashboard
- AWS Health Dashboard
- Ping / Traceroute



NOTE

A single VPN tunnel failure should not impact your business if redundancy and routing are properly configured.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA



AWS WAF BLOCKING LEGITIMATE USERS

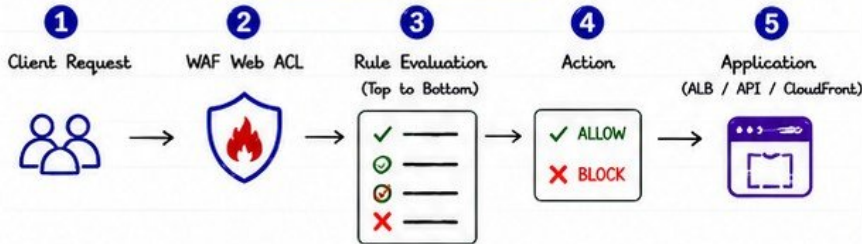
Legitimate users are blocked by AWS WAF rules, causing errors, login failures, or API issues.



AT A GLANCE

- ✓ Rule evaluation order
- ✓ Managed rules tuning
- ✓ Identify false positives
- ✓ Traffic analysis & verification
- ✓ Allow legitimate traffic safely

HOW AWS WAF WORKS



If a rule matches and action = **BLOCK** → request is blocked.

RULE EVALUATION ORDER

- 1 AWS WAF processes rules in priority order (lowest number first).
- 2 First match takes action (allow, block, count).
- 3 If no rule matches → default action applies.
- 4 Rule groups follow the order defined.
- 5 Managed rule groups evaluate their rules in their own order.
- 6 Labels from matched rules can be used by later rules.

COMMON REASONS LEGITIMATE USERS ARE BLOCKED



MANAGED RULES TOO STRICT

Default settings may block valid requests (e.g., SQLi, XSS, Bot Control).



IP REPUTATION LISTS

IP in bad reputation list even if traffic is legitimate.



GEO / RATE LIMIT RULES

Geolocation or rate limits blocking users from allowed regions or burst traffic.



CUSTOM RULES MISCONFIGURED

Regex, size constraints, or header rules are too aggressive.

MANAGED RULE GROUPS (TUNE FOR YOUR APP)

- ✓ AWSManagedRulesCommonRuleSet
- ✓ AWSManagedRulesKnownBadInputsRuleSet
- ✓ AWSManagedRulesSQLiRuleSet
- ✓ AWSManagedRulesXSSRuleSet
- ✓ AWSManagedRulesBotControlRuleSet
- ✓ AWSManagedRulesAnonymousIpList
- ✓ AWSManagedRulesAmazonIpReputationList



Start in **COUNT** mode to monitor impact before switching to **BLOCK**.

INVESTIGATION WORKFLOW

- 1 Confirm user impact → Check apps, ALB/CloudFront 4xx/5xx
- 2 Check WAF metrics → `aws wafv2 get-metric-data`
- 3 Review sampled requests → `aws wafv2 get-sampled-requests`
- 4 Analyze blocked requests → `aws wafv2 get-logging-configuration`
- 5 Identify matching rule → Check ruleId in logs / samples
- 6 Validate rule conditions → Test rule conditions with user request
- 7 Adjust rule action → Set to COUNT or add exception
- 8 Monitor and re-evaluate → Confirm users can access, then optimize

TRAFFIC ANALYSIS CHECKLIST

- ☐ Check source IP and geo.
- ☐ Validate user agent and headers.
- ☐ Review rate limiting thresholds.
- ☐ Inspect request size and patterns.
- ☐ Compare blocked vs allowed samples.
- ☐ Use CloudWatch + logs for trends.
- ☐ Correlate with business events.



Use logs + samples to understand WHY the rule matched.

EXAMPLE: BLOCKED LOG (JSON)

```

{
  "timestamp": "2024-05-15T12:34:56Z",
  "terminatingRuleId": "AWSManagedRulesCommonRuleSet",
  "action": "BLOCK",
  "httpRequest": {
    "clientIp": "203.0.113.45",
    "url": "/login",
    "args": "user=ann&pass=1234"
  },
  "labels": ["aws:waf:managed:aws:common-ruleset:generic-sqli"]
}
  
```



Focus on **terminatingRuleId** to find the rule causing the block.

RESOLUTION OPTIONS

- ✓ Change rule action to COUNT first.
- ✓ Add exceptions for specific IPs or paths.
- ✓ Adjust rate limits or geo restrictions.
- ✓ Refine custom rule conditions (regex, size, headers).
- ✓ Use labels and scope-down statements.
- ✓ Update AWS managed rule group settings.
- ✓ Continuously monitor after changes.



BEST PRACTICES

- ✓ Start new rules in COUNT mode.
- ✓ Use rule groups and labels effectively.
- ✓ Keep rules simple and specific.
- ✓ Regularly review WAF logs & metrics.
- ✓ Tune managed rule groups for your app.
- ✓ Have an exception process for legit traffic.



USEFUL TOOLS & DASHBOARDS



CloudWatch Metrics (WAF)
WAF Logs in CloudWatch Logs
WAF Sampled Requests
AWS Managed Rules Dashboard
X-Ray / ALB Logs (for correlation)

REMEMBER

WAF is your shield, but tuning it right ensures protection without blocking legitimate users.

SECURE YOUR APP ★ PROTECT USERS ★ TUNE SMARTLY ★ MONITOR CONTINUOUSLY



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA



GUARDDUTY DETECTS CRITICAL THREAT

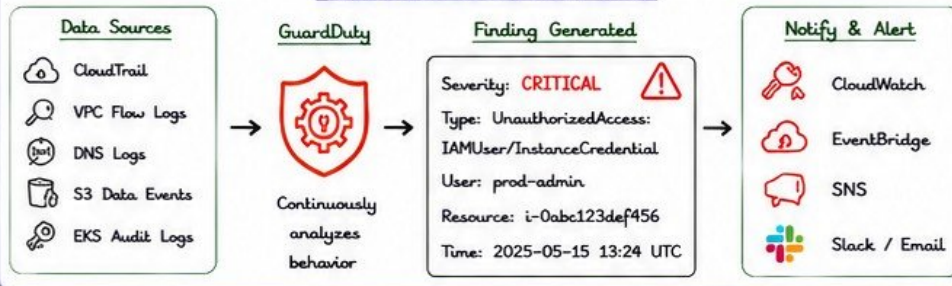
AWS GuardDuty detected a critical threat indicating potential compromise of your AWS environment. Immediate investigation and response are required.



AT A GLANCE

- ✓ Detect & prioritize threat
- ✓ Investigate findings deeply
- ✓ Identify compromised assets
- ✓ Contain & eradicate threat
- ✓ Recover & harden environment

GUARDDUTY DETECTION FLOW

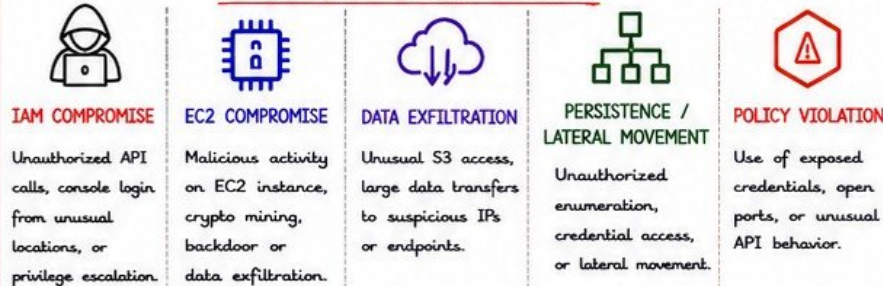


WHAT IS GUARDDUTY?

AWS GuardDuty is a threat detection service that uses ML, anomaly detection and threat intel to identify malicious or unauthorized activity in your AWS environment.

It helps you detect threats you might otherwise miss.

COMMON CRITICAL FINDING TYPES



INVESTIGATION CHECKLIST

- ☐ Review GuardDuty finding details
- ☐ Identify affected account, user, resource
- ☐ Check CloudTrail events
- ☐ Review VPC Flow Logs
- ☐ Validate IAM user / role activity
- ☐ Check EC2 instance status & logs
- ☐ Identify indicators of compromise (IOC)
- ☐ Determine scope of impact
- ☐ Prioritize containment actions
- ☐ Document findings & timeline
- ☐ Report & escalate if needed

Use Detective / Security Hub for deeper investigation and correlation.

THREAT INVESTIGATION WORKFLOW

- 1 Review GuardDuty Finding → Check severity, type, resource, time
- 2 Identify Affected Resource → User, Role, EC2, S3, IP, etc.
- 3 Analyze CloudTrail Events → aws cloudtrail lookup-events
- 4 Check VPC Flow Logs → Analyze connections & data transfer
- 5 Validate IAM Activity → aws iam get-user / get-policy
- 6 Inspect EC2 Logs → /var/log/auth.log, syslog, app logs
- 7 Search for IOCs → Suspicious IPs, domains, hashes
- 8 Determine Impact → Data access, privilege escalation, etc.
- 9 Contain & Remediate → Disable access, terminate, update policies
- 10 Monitor & Harden → Watch metrics, update detections

IAM COMPROMISE INVESTIGATION

- ✓ Check unusual console sign-ins
- ✓ Review API calls for privilege escalation
- ✓ Identify new or modified access keys
- ✓ Check role assumption events
- ✓ Validate MFA usage
- ✓ Disable compromised credentials
- ✓ Rotate access keys and passwords
- ✓ Revoke active sessions
- ✓ Least privilege review
- ✓ Monitor for recurring activity

Focus on who, when, where and what was accessed.

RESPONSE WORKFLOW



EC2 COMPROMISE CHECKLIST

- ✓ Check GuardDuty findings related to the instance
- ✓ Verify instance state and network connectivity
- ✓ Review system logs: authlog, cloud-init, application logs
- ✓ Check running processes and startup scripts
- ✓ Scan for suspicious files / cron jobs
- ✓ Check security group rules
- ✓ Isolate instance (stop / detach network)
- ✓ Rebuild instance from known good AMI if required
- ✓ Rotate credentials stored on the instance

CONTAINMENT ACTIONS

- ✓ Disable IAM user / access keys
- ✓ Revoke active sessions
- ✓ Detach compromised role policies
- ✓ Stop or isolate EC2 instance
- ✓ Block malicious IPs in NACL / SG
- ✓ Enable MFA and access tokens
- ✓ Snapshot disk for forensics

BEST PRACTICES

- ✓ Enable GuardDuty in all regions & accounts
- ✓ Use Organization-wide coverage
- ✓ Integrate with Security Hub & SIEM
- ✓ Review findings daily
- ✓ Automate response with EventBridge
- ✓ Keep systems & dependencies updated
- ✓ Least privilege for all identities

KEY CLOUDWATCH METRICS

- GuardDuty Findings (findingSeverity)
- UnauthorizedAccess (Count)
- EC2 Status Checks (Failed)
- NetworkPacketIn / Out
- API Call Rate (CloudTrail)

NOTE

A critical finding is not always an active breach, but it is always a signal. Investigate quickly, contain decisively, and harden continuously.



USEFUL TOOLS

- GuardDuty
- AWS Detective
- Security Hub
- CloudTrail
- VPC Flow Logs
- AWS Config



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA



SECURITY HUB REPORTS

MULTIPLE CRITICAL FINDINGS



AT A GLANCE

- ✓ Aggregate & centralize findings
- ✓ Prioritize critical risks
- ✓ Map to compliance standards
- ✓ Remediate & track progress
- ✓ Continuously improve posture

AWS Security Hub aggregates findings from AWS services, partner products and custom checks to give you a comprehensive view of security posture and help you prioritize remediation.

HOW SECURITY HUB WORKS

1. DATA SOURCES
2. AGGREGATION
3. ANALYSIS
4. PRIORITIZATION
5. RESPONSE



- AWS Services
- Partner Products
- Custom Insights
- CIS Benchmarks



Security Hub aggregates & normalizes findings



Findings are enriched with context, severity and compliance mapping



Critical findings are prioritized based on risk & impact



Investigate, remediate & monitor continuously

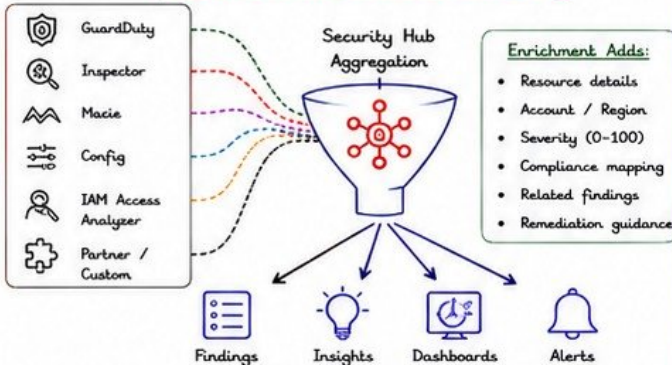
WHAT IS AWS SECURITY HUB?

Security Hub gives you a comprehensive view of your security state within AWS by aggregating, organizing and prioritizing security findings.

Benefits:

- Centralized security visibility
- Automated aggregation & deduplication
- Risk-based prioritization
- Compliance standards mapping
- Remediation tracking

FINDING AGGREGATION & ENRICHMENT



FINDING PRIORITIZATION

Findings are scored and prioritized based on risk.

SEVERITY	SCORE RANGE	DESCRIPTION	ACTION
CRITICAL	70 - 100	High risk, immediate impact possible	Immediate Remediation
HIGH	40 - 69	Significant risk, likely impact	Fix soon
MEDIUM	20 - 39	Moderate risk, limited impact	Plan & Fix
LOW	0 - 19	Low risk, minimal impact	Monitor

★ Focus first on **CRITICAL**, then **HIGH** findings.

INVESTIGATION WORKFLOW

- 1 Review Security Hub Dashboard → Open dashboards & summary
- 2 Review Critical Findings → Filter by Severity = Critical
- 3 Investigate Finding Details → Check resource, account, region
- 4 Analyze Related Findings → Look for patterns & correlations
- 5 Map to Compliance Standards → Use AWS Standards / Frameworks
- 6 Validate Impact & Exploitability → Assess real-world risk
- 7 Create / Update Remediation Plan → Assign owner & due date
- 8 Track & Verify Remediation → Confirm resolved & re-test



Use insights & custom views to speed up investigation.

COMPLIANCE MAPPING

Findings are mapped to standards:

- ✓ CIS AWS Foundations Benchmark
- ✓ NIST 800-53 / 800-171
- ✓ PCI DSS
- ✓ ISO 27001
- ✓ SOC 2
- ✓ Custom Standards



Helps you stay audit-ready and prove security posture.

REMEDIATION EXAMPLES

- IAM USER WITH ADMIN ACCESS → Apply least privilege. Remove unnecessary permissions
- SECURITY GROUP OPEN TO WORLD → Restrict CIDR ranges. Use specific IPs
- S3 BUCKET PUBLIC READ → Block Public Access. Enable encryption
- EC2 INSTANCE NO PATCHES → Apply patches. Enable SSM



★ Remediate → Verify → Monitor → Repeat

SECURITY HUB INSIGHTS

Insights group related findings to reduce noise.

Examples:

- Unused Security Groups
- S3 Buckets with Public Access
- Root Account Usage
- MFA not enabled



DASHBOARD METRICS

- Total Findings
- Findings by Severity
- Findings by Resource Type
- Top 10 Resources by Findings
- Compliance Score
- Trending Findings



BEST PRACTICES

- ✓ Enable all relevant standards & controls
- ✓ Review findings daily
- ✓ Automate remediation where possible
- ✓ Use insights to reduce noise
- ✓ Set up EventBridge alerts
- ✓ Track remediation until the end
- ✓ Continuously improve your posture



USEFUL TOOLS

- Security Hub Console
- AWS CLI (aws securityhub)
- EventBridge
- AWS Config
- CloudWatch
- Systems Manager

NOTE

A finding is only useful if you act on it. Prioritize, remediate and improve continuously.



Instagram: @veriqta



LinkedIn: VERIQTA



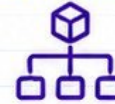
X: @veriqta



YouTube: VERIQTA



AWS ORGANIZATIONS SCP BLOCKING DEPLOYMENTS



A deployment fails because an AWS Organizations Service Control Policy (SCP) explicitly denies the required action, even though the IAM user has permissions.

AT A GLANCE

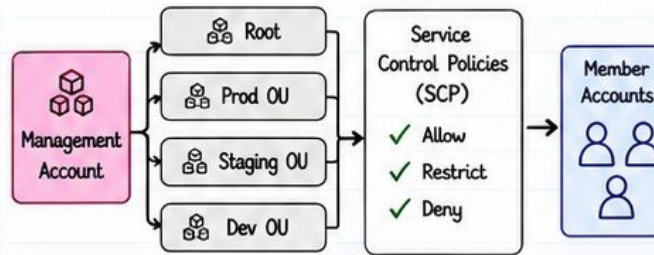
- ✓ Understand SCP effect
- ✓ Review effective permissions
- ✓ Identify explicit deny
- ✓ Fix at the right OU level
- ✓ Restore deployment safely

WHAT IS AWS ORGANIZATIONS?

AWS Organizations helps you centrally manage multiple AWS accounts at scale.

SCPs set the maximum permissions available in each account.

HOW SCP WORKS



SCP VS IAM POLICIES

Feature	SCP	IAM Policy
Purpose	Sets max permissions for accounts	Grants permissions to users / roles within an account
Effect	Can only ALLOW or DENY	Can ALLOW or DENY
Scope	Account / OU	User / Role / Resource
Evaluated by	AWS Org	IAM

IAM EVALUATION LOGIC (with SCP)

- 1 Start with implicit deny.
- 2 Check SCP → if explicit **DENY**, request is **DENIED**.
- 3 If allowed by SCP → check Permission Boundary (if attached).
- 4 Then evaluate IAM Policies.
- 5 Then evaluate Resource-based Policies.
- 6 Final decision = most restrictive (explicit DENY wins).



COMMON DEPLOYMENT FAILURE SCENARIOS

CloudFormation	EC2	IAM	S3
DENIED	DENIED	DENIED	DENIED
e.g. cloudformation:* blocked	e.g. RunInstances blocked	e.g. iam:CreateRole blocked	e.g. s3:PutObject blocked

INVESTIGATION CHECKLIST

- ☐ Confirm account is in AWS Organizations
- ☐ Identify applied SCPs (Root / OU)
- ☐ Use aws organizations describe-effective-policy
- ☐ Check CloudTrail for AccessDenied
- ☐ Verify permission boundaries
- ☐ Review IAM role / user policies
- ☐ Map which OU is blocking
- ☐ Test in another account / OU



Always find the explicit deny.

TROUBLESHOOTING STEPS

- 1 Check the deployment error → Look for AccessDenied / explicit deny
- 2 Get effective SCP → aws organizations describe-effective-policy
- 3 Decode the deny → Identify the blocked action
- 4 Check OU hierarchy → Verify which SCP is applied
- 5 Review permission boundaries → Ensure no extra restriction
- 6 Check CloudTrail → Confirm the exact denied action
- 7 Work with Org admin → Update SCP to allow required action
- 8 Retest deployment → Confirm success



If it works in one account but fails in another, it is usually an SCP.

EXAMPLE ERROR

An error occurred (AccessDenied) when calling the RunInstances operation:
User is not authorized to perform: ec2:RunInstances because no identity-based policy allows the ec2:RunInstances action and an explicit deny exists in a service control policy

REMEDIATION OPTIONS

- ✓ Modify SCP at the correct OU.
- ✓ Allow specific service / action.
- ✓ Use conditions for fine-grained control.
- ✓ Avoid using "Deny *" unless necessary.
- ✓ Test in a sandbox account before production.



SAMPLE SCP (ALLOW)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:RunInstances",
        "cloudformation:*"
      ],
      "Resource": "*"
    }
  ]
}
```



BEST PRACTICES

- ✓ Use SCPs to restrict, not break development.
- ✓ Document and communicate SCP rules.
- ✓ Use least privilege.
- ✓ Use conditions instead of broad denies.
- ✓ Keep a separate OU for dev / test.
- ✓ Monitor changes with CloudTrail.
- ✓ Add exceptions for pipeline roles.



USEFUL TOOLS

- AWS Organizations Console
- AWS CLI (organizations)
- IAM Policy Simulator
- CloudTrail
- AWS Config
- Amazon EventBridge

KEY TAKEAWAY

- SCPs set the maximum permissions for an account.
- Even with full IAM permissions, an explicit deny in an SCP will always block your actions.
- Find the deny. Fix at the right OU. Deploy safely.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA



CROSS-ACCOUNT IAM ROLE ASSUMPTION FAILURE

TRUST POLICIES • EXTERNAL IDS • ROLE CHAINING • STS TROUBLESHOOTING



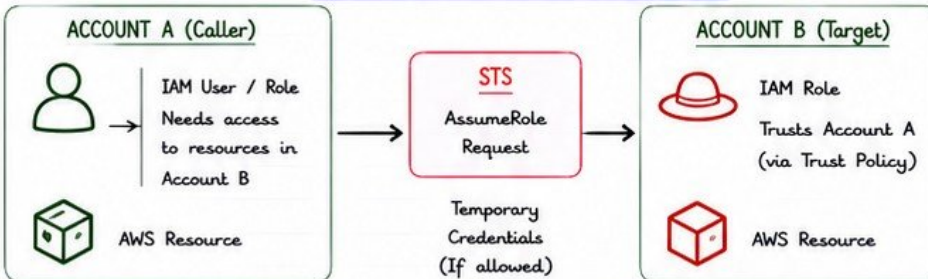
AT A GLANCE

- ✓ Fix trust policy issues
- ✓ Validate External IDs
- ✓ Understand role chaining
- ✓ Troubleshoot with STS
- ✓ Restore secure access

Cross-account access breaks when IAM roles cannot be assumed due to trust policy mismatches, missing External IDs, permission boundaries or role chaining issues.



CROSS-ACCOUNT ASSUMPTION FLOW



WHY ASSUMPTION FAILS

- ✗ Trust policy does not allow the caller
- ✗ External ID is missing or incorrect
- ✗ MFA required but not provided
- ✗ Permission boundary blocks the action
- ✗ Role chaining limit exceeded
- ✗ Session policy is too restrictive
- ✗ Invalid or expired credentials
- ✗ SCP or Resource policy denies access

TRUST POLICY EXAMPLE (ACCOUNT B ROLE)

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::111122223333:root"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "VERIQTA-12345"
        }
      }
    }
  ]
}
  
```

KEY POINTS

- Principal must match the caller's account.
- ExternalId must match exactly.
- Conditions must be satisfied.

EXTERNAL ID (CONFUSED DEPUTY PROTECTION)

External ID ensures only your account can assume the role and prevents other customers from doing the same.

Best Practice:

- ✓ Always require External ID for 3rd party access
- ✓ Use unique External ID per customer
- ✓ Store External ID securely (SSM Parameter Store or Secrets Manager)

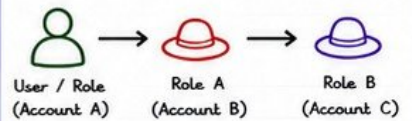
Example CLI

```

aws sts assume-role \
--role-arn arn:aws:iam::444455556666:role/TargetRole \
--role-session-name VeriqtaSession \
--external-id VERIQTA-12345
  
```

ROLE CHAINING

A user/role assumes Role A, then uses those temporary credentials to assume Role B (in same or another account).



IMPORTANT LIMITS

- Max 1 hour session if role chaining.
- Cannot use MFA again when chaining.
- Keep chain as short as possible.

STS TROUBLESHOOTING COMMANDS

1	aws sts get-caller-identity	Verify current identity
2	aws sts assume-role	Attempt role assumption
3	aws sts decode-authorization-message	Decode AccessDenied messages
4	aws iam simulate-principal-policy	Simulate permissions
5	aws iam get-role	Check role configuration
6	aws iam get-role-policy	Check inline policies
8	aws iam list-attached-role-policies	List attached policies



Decode error messages for the real reason using `decode-authorization-message`.

COMMON ERROR MESSAGES

ERROR	CAUSE
AccessDenied	Not authorized by trust policy or permissions
ExternalIdMismatch	Missing or incorrect External ID
MFARequired	MFA required but not provided
MalformedPolicyDocument	Invalid trust policy syntax
PackedPolicyTooLarge	Session policy too large
ExpiredToken	Temporary credentials expired



Decode the message for exact reason!

INVESTIGATION CHECKLIST

- ☐ Confirm caller account ID and role/user ARN
- ☐ Review target role trust policy
- ☐ Check External ID requirement
- ☐ Validate permissions in caller identity
- ☐ Check permission boundaries (caller & target)
- ☐ Check SCPs in AWS Organizations
- ☐ Verify session duration & role chaining
- ☐ Check MFA requirement
- ☐ Test with `sts assume-role` CLI
- ☐ Decode error message for details



Start with trust policy, External ID and permissions.

BEST PRACTICES

- ✓ Use least privilege for cross-account roles
- ✓ Always use External IDs for 3rd party access
- ✓ Keep trust policies specific (no wildcards *)
- ✓ Set appropriate session duration
- ✓ Monitor usage with CloudTrail
- ✓ Rotate and audit External IDs



USEFUL TOOLS

- AWS STS
- IAM Policy Simulator
- AWS CLI
- CloudTrail
- IAM Access Analyzer
- Access Advisor

CLOUDTRAIL CLUE

Look for AssumeRole events in CloudTrail. Check source IP, `userIdentity` and `errorCode`.



KEY TAKEAWAY

Cross-account access depends on three things working together:

- Trust Policy (who can assume)
- External ID (who is authorized)
- Permissions (what they can do)

Fix the root cause, not just the symptom!



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

INCIDENT #40



VERIQTA AWS BACKUP RESTORE FAILED



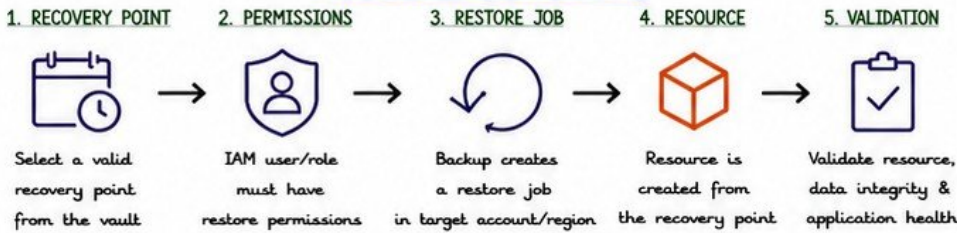
A restore operation fails due to missing permissions, invalid recovery point, incorrect settings, or cross-region limitations.

Fast recovery depends on correct configuration and validation.

AT A GLANCE

- ✓ Verify recovery points
- ✓ Check vault & IAM policies
- ✓ Confirm region & encryption
- ✓ Execute restore correctly
- ✓ Validate recovery & data

AWS BACKUP RESTORE FLOW



WHY RESTORE FAILS

- ✗ No valid recovery points available
- ✗ Insufficient IAM permissions
- ✗ Vault policy denies restore
- ✗ KMS key permissions missing
- ✗ Wrong region or account
- ✗ Encrypted backups with no access
- ✗ Service limits or quotas exceeded
- ✗ Dependent resources not restored
- ✗ Corrupted or incomplete backup

RECOVERY POINTS

- Immutable backups stored in AWS Backup vaults
- Each recovery point has:
 - Recovery Point ARN
 - Resource type
 - Creation time
 - Encryption key
 - Lifecycle status (cold/expired)
- Check availability:



```
aws backup list-recovery-points-by-backup-vault \
--backup-vault-name MyVault
```

VAULT POLICIES & PERMISSIONS

Vault Access Policy

Must allow backup:Restore action for the user/role.

```
{
  "Effect": "Allow",
  "Action": [
    "backup:RestoreJob*",
    "backup:GetRecoveryPoint*",
    "backup:ListRecoveryPoints*"
  ],
  "Resource": "*"
}
```

IAM Permissions

User/role needs these minimum permissions:

- ✓ backup:StartRestoreJob
- ✓ backup:GetRecoveryPoint
- ✓ backup:RestoreJob
- ✓ kms:Decrypt (if encrypted)
- ✓ iam:PassRole (if using service role)



CROSS-REGION RESTORE



- Backups are region-specific.
- To restore in another region:
 - 1 Copy recovery point to destination region
 - 2 Ensure vault exists in target region
 - 3 Start restore in target region
 - 4 Check KMS key availability in target region
- Note:** Some resources (e.g. AMIs) must be copied before restore.

RESTORE JOB STATES

STATUS	DESCRIPTION
PENDING	Job is accepted and waiting
RUNNING	Restore is in progress
COMPLETED	Restore finished successfully
PARTIALLY_COMPLETED	Some resources restored
FAILED	Restore failed
EXPIRED	Recovery point expired
CANCELED	Restore job was canceled

COMMON RESTORE FAILURE REASONS

- ⚠ AccessDeniedException (Permissions or vault policy)
- ⚠ InvalidRecoveryPointException (Recovery point not found/expired)
- ⚠ KMSAccessDeniedException (Cannot decrypt backup)
- ⚠ ResourceLimitExceededException (Service quota limits)
- ⚠ InvalidParameterValueException (Wrong region/account)
- ⚠ DependentFailure (Missing dependent resources)
- ⚠ UnsupportedRestoreException (Resource type not supported)

INVESTIGATION WORKFLOW

- 1 Identify failed restore job → `aws backup list-restore-jobs`
- 2 Check job details & error → `aws backup describe-restore-job`
- 3 Verify recovery point → `aws backup describe-recovery-point`
- 4 Validate vault permissions → Check vault access policy
- 5 Check IAM permissions → Use IAM Policy Simulator
- 6 Verify KMS key access → Check key policy & grants
- 7 Confirm region/account → Ensure correct target
- 8 Check quotas & limits → Service Quotas console
- 9 Review CloudTrail events → Filter: backup.amazonaws.com

RESTORE VALIDATION CHECKLIST

- ☐ Resource exists and is in expected state
- ☐ Configuration matches source environment
- ☐ Data integrity verified (checksums / counts)
- ☐ Application starts successfully
- ☐ Networking & dependencies are functional
- ☐ Security groups & IAM roles are correct
- ☐ Encryption & keys are accessible
- ☐ Monitor logs and metrics after restore



BEST PRACTICES

- ✓ Test restores regularly in a sandbox account
- ✓ Use backup plans with lifecycle & retention
- ✓ Enable cross-region copy for critical backups
- ✓ Use customer-managed KMS keys with proper access
- ✓ Document restore runbooks
- ✓ Tag backups and resources consistently
- ✓ Monitor backup & restore jobs with alarms



USEFUL COMMANDS

```
List recovery points
aws backup list-recovery-points-by-backup-vault

Describe recovery point
aws backup describe-recovery-point --recovery-point-arn <arn>

Start restore job
aws backup start-restore-job --recovery-point-arn <arn>

Describe restore job
aws backup describe-restore-job --restore-job-id <id>

List restore jobs
aws backup list-restore-jobs
```



KEY TAKEAWAY

Backups are only as good as your ability to restore them. Understand permissions, policies, regions, and dependencies. Test restores. Validate recovery. Stay prepared.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA



EFS PERFORMANCE ISSUES

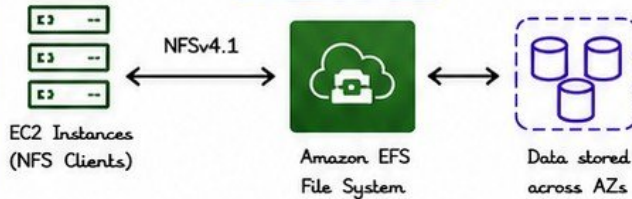


AT A GLANCE

- ✓ Understand throughput modes
- ✓ Monitor burst credits
- ✓ Optimize NFS clients
- ✓ Tune performance settings
- ✓ Validate improvements

Amazon EFS is a managed NFS file system designed to scale. Performance issues usually occur due to incorrect throughput mode, burst credit depletion, network latency, or too many NFS clients.

HOW EFS WORKS



- EFS uses the NFSv4.1 protocol.
- Data is automatically replicated across AZs.
- Multiple EC2 instances can mount the file system concurrently.
- Scales automatically, but performance depends on configuration.

THROUGHPUT MODES

Choose the right mode for your workload.

Bursting



Baseline throughput + burst capacity using credits.

Best for:
General purpose workloads.

Provisioned



You choose the throughput (MiB/s).

Ideal for predictable high-performance workloads.

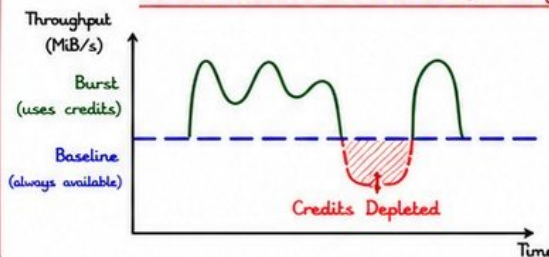
Elastic



Automatically scales throughput with amount of storage.

Best for:
Unpredictable, growing workloads.

BURST CREDITS EXPLAINED (Bursting Mode)



Credit Rules

- Earn credits when using less than baseline.
- Spend credits when bursting.
- When credits are 0, throughput drops to baseline.

COMMON SYMPTOMS

- Slow file read/write operations
- High latency in application
- Throughput stuck at baseline
- Burst credits at or near zero
- Timeouts or NFS errors (stale file handle, timeout)



INVESTIGATION WORKFLOW

- 1 Check EFS metrics → CloudWatch: Throughput, BurstCreditBalance
- 2 Check throughput mode → `aws efs describe-file-systems`
- 3 Check burst credits → BurstCreditBalance metric
- 4 Check latency → CloudWatch: ClientConnections, DataReadIOBytes
- 5 Check NFS clients → List mounts and client count
- 6 Verify network path → Check SG, NACL, ENI performance
- 7 Identify workload pattern → Read/write mix, file ops/sec
- 8 Reproduce & validate → Test after tuning changes



Always correlate CloudWatch metrics with application behavior.

PERFORMANCE TUNING CHECKLIST

- ☐ Choose the correct throughput mode
- ☐ If Bursting → monitor and manage burst credits
- ☐ If Provisioned → set adequate throughput (MiB/s)
- ☐ Use multiple mount targets (one per AZ)
- ☐ Spread NFS clients across AZs
- ☐ Use NFSv4.1
- ☐ Enable TLS if security is required (note: slight overhead)
- ☐ Optimize application I/O and file operations
- ☐ Avoid small random writes where possible
- ☐ Monitor continuously and adjust as needed



USEFUL COMMANDS

- ```
Describe file system
aws efs describe-file-systems

Describe mount targets
aws efs describe-mount-targets --file-system-id fs-xxxx

CloudWatch metrics (example)
aws cloudwatch get-metric-statistics \
 --namespace AWS/EFS \
 --metric-name ThroughputInBytes \
 --dimensions Name=FileSystemId,Value=fs-xxxx \
 --start-time <start> --end-time <end> --period 300 --statistics Average

Check burst credits
aws cloudwatch get-metric-statistics \
 --namespace AWS/EFS --metric-name BurstCreditBalance \
 --dimensions Name=FileSystemId,Value=fs-xxxx \
 --start-time <start> --end-time <end> --period 300 --statistics Average
```



## BEST PRACTICES

- ✓ Right-size throughput mode based on workload.
- ✓ Monitor burst credit balance and alarms.
- ✓ Use one mount target per AZ.
- ✓ Distribute clients across AZs.
- ✓ Use NFSv4.1 and enable sessions for better performance.
- ✓ Automate CloudWatch alarms on credit balance & throughput.
- ✓ Regularly review and tune as workload grows.



## KEY METRICS (CloudWatch)



**ThroughputInBytes**  
Total data read + written per second.



**BurstCreditBalance**  
Current burst credits available.



**ClientConnections**  
Number of connected NFS clients.



**DataReadIOBytes / DataWriteIOBytes**  
Read/write I/O in bytes.



**PercentIOLimit**  
Utilization of throughput limit.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA





# FSx STORAGE PERFORMANCE BOTTLENECK



## AT A GLANCE

- ✓ Analyze IOPS & throughput
- ✓ Understand file system types
- ✓ Tune or scale performance
- ✓ Monitor & validate changes
- ✓ Prevent future bottlenecks

Amazon FSx delivers fully managed, high-performance file systems. Performance bottlenecks occur when workloads exceed provisioned capacity, are misconfigured, or don't match the right file system.

## FSx OVERVIEW

AWS FSx provides different file systems optimized for various workloads.



FSx for Windows File Server

SMB protocol, Windows ACLs, broad Windows compatibility.

FSx for FSx

High performance HPC workloads, Linux clients.

FSx for NetApp ONTAP



Enterprise features, snapshots, FlexVols, multi-protocol.

FSx for OpenZFS



Open source ZFS compatibility, Linux based.

## COMMON SYMPTOMS

- ⚠ High latency in file operations
- ⚠ Low IOPS / Throughput
- ⚠ Slow application or user experience
- ⚠ NFS / SMB timeouts
- ⚠ Queue depth consistently high
- ⚠ Burst credits depleted (FSx for Windows/Lustre)
- ⚠ CloudWatch metrics trending at limit



## PERFORMANCE FACTORS

Capacity Limits



- Provisioned IOPS
- Provisioned Throughput
- Burst balance / credits
- File system size

Workload Pattern



- Small file operations
- Large sequential I/O
- Random vs sequential
- Concurrency

Client Configuration



- NFS rsize / wsize
- NFS version (v3/v4.1)
- SMB Multichannel
- Mount options

Network



- VPC bandwidth
- AZ placement
- Network latency
- Packet loss

## WINDOWS vs LUSTRE

FSx for Windows File Server



- ✓ SMB protocol
- ✓ Windows ACLs
- ✓ Supports DFS, AD
- ✓ Burst credits based
- ✓ Great for enterprise apps
- ✓ Lower latency for SMB clients

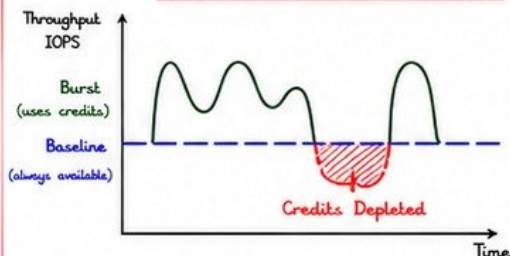
FSx for Lustre



- ✓ NFS protocol
- ✓ High throughput & IOPS
- ✓ Designed for HPC
- ✓ No burst credits (steady perf.)
- ✓ Best for large sequential I/O
- ✓ Linux clients only

VS

## BURST CREDITS (Windows & Lustre)



### Credit Rules

- Earn credits when using less than baseline.
- Spend credits when bursting.
- When credits are 0, throughput drops to baseline.



## INVESTIGATION WORKFLOW

- 1 Check CloudWatch metrics → FSx metrics: DataReadIOPS, DataWriteIOPS, Throughput, BurstCreditBalance
- 2 Check file system limits → `aws fsx describe-file-systems`
- 3 Check client connections → NFS/SMB sessions, mount options
- 4 Review application behavior → Identify heavy I/O workloads
- 5 Check network performance → VPC flow logs, latency, bandwidth
- 6 Compare with baseline → Identify trend vs normal
- 7 Identify root cause → Capacity, config, workload or network
- 8 Apply optimization → Tune or scale based on findings



Always correlate metrics with application behavior.

## PERFORMANCE TUNING STRATEGIES

- ✓ Increase provisioned IOPS / Throughput
- ✓ Right-size file system for workload
- ✓ Enable / monitor burst credits
- ✓ Use appropriate file system type
- ✓ Tune NFS mount options (rsize / wsize)
- ✓ Enable SMB Multichannel (Windows)
- ✓ Distribute workload across multiple clients
- ✓ Optimize application I/O pattern
- ✓ Place clients in same AZ for lower latency

### Scaling Strategy

- Vertical scaling: Increase IOPS/Throughput on same file system.
- Horizontal scaling: Use additional file systems and distribute workload.
- Multi-AZ access: Use preferred AZ or equally positioned clients.

## USEFUL COMMANDS

- `aws fsx describe-file-systems`
- `aws fsx describe-metrics`
- `aws fsx describe-tags --resource-id <fs-id>`
- `aws cloudwatch get-metric-statistics \`  
`--namespace AWS/FSx \`  
`--metric-name DataReadIOPS \`  
`--dimensions Name=FileSystemId,Value=<fs-id> \`  
`--start-time <start> --end-time <end> \`  
`--period 300 --statistics Average,Maximum`
- `mount | grep nfs`
- `nfsstat -m`

## KEY METRICS (CloudWatch)



DataReadIOPS / DataWriteIOPS  
Read/Write operations per second.



DataReadThroughput / DataWriteThroughput  
Throughput in MB/s.



BurstCreditBalance  
Available burst credits.



ClientConnections  
Number of active client connections.

## BEST PRACTICES

- ✓ Monitor metrics and set alarms
- ✓ Understand workload requirements
- ✓ Use right file system for the job
- ✓ Plan capacity ahead of time
- ✓ Test performance after changes



## KEY TAKEAWAY

FSx performance depends on the right file system choice, proper sizing, client configuration, and continuous monitoring. Tune or scale early to avoid impact on users and applications.



## INVESTIGATION CHECKLIST

- ☐ Metrics reviewed and correlated
- ☐ Burst credits healthy
- ☐ Mount options optimized
- ☐ Network performance verified
- ☐ Root cause identified
- ☐ Performance improved & validated



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA



## INCIDENT #43



VERIQTA

# DYNAMODB THROTTLING



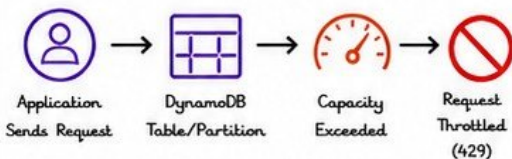
## AT A GLANCE

- ✓ Identify throttling & hot partitions
- ✓ Optimize key design & access patterns
- ✓ Enable Auto Scaling
- ✓ Monitor & alert continuously
- ✓ Plan capacity for predictable growth

DynamoDB throttling occurs when requests exceed the provisioned capacity for a table or a partition. It leads to failed requests, higher latency, and degraded application performance.

### WHAT IS THROTTLING?

DynamoDB returns `ProvisionedThroughputExceededException` when the table or a partition exceeds its capacity.



### COMMON SYMPTOMS

- ⚠ `ProvisionedThroughputExceededException` errors
- ⚠ High `ThrottledRequests` in CloudWatch
- ⚠ Increased latency for reads/writes
- ⚠ Some items succeed, some fail
- ⚠ Particular partitions (keys) are hot
- ⚠ Application retries increase traffic

### THROTTLING LEVELS

Throttling can occur at two levels:



**Table Level Throttling**  
Total read/write capacity is exceeded across the entire table.



**Partition Level Throttling**  
A single partition (key range) receives more traffic than it can handle (hot partition).

### DYNAMODB CAPACITY MODES

#### PROVISIONED MODE



- You specify RCU & WCU
- Predictable performance
- Supports Auto Scaling
- Best for steady workloads

#### ON-DEMAND MODE



- No capacity planning required
- Pay per request
- Handles unpredictable traffic
- Higher cost for steady workloads

### PARTITION KEYS & HOT PARTITIONS

#### GOOD KEY DESIGN

- ✓ Distributes requests evenly
- ✓ High cardinality
- ✓ Avoid sequential values
- ✓ Example: `user_id`, `order_id`

#### GOOD EXAMPLE

Partition Key: `user_id`  
`user_001`, `user_002`, `user_003`  
`user_004`, `user_005` ...  
 (traffic spread evenly)

#### BAD KEY DESIGN (HOT PARTITION)

- ✗ Low cardinality
- ✗ Sequential or time-based keys
- ✗ Many requests hit same key
- ✗ Example: `status`, `date`

#### BAD EXAMPLE

Partition Key: `status`  
`status = 'ACTIVE'` for all items  
 (all traffic hits one partition)

### CLOUDWATCH METRICS TO WATCH



- `ConsumedReadCapacityUnits`
- `ConsumedWriteCapacityUnits`
- `ProvisionedReadCapacityUnits`
- `ProvisionedWriteCapacityUnits`
- `ThrottledRequests`
- `SuccessfulRequestLatency`
- `ReturnedItemCount`

### INVESTIGATION WORKFLOW

- 1 Check CloudWatch metrics → `ThrottledRequests`, `Consumed*`
- 2 Identify throttling level → Table level or partition level?
- 3 Find hot partitions → `Contributor Insights`
- 4 Review access patterns → Which keys are most active?
- 5 Check item size → Large items consume more RCUs
- 6 Evaluate capacity mode → Provisioned vs On-Demand
- 7 Optimize or scale → Repartition, index, or Auto Scaling
- 8 Test & monitor → Validate improvement



Always correlate metrics with application behavior.

### AUTOSCALING (PROVISIONED MODE)

Enable Auto Scaling for:

- ✓ Read Capacity (RCU)
- ✓ Write Capacity (WCU)

Auto Scaling settings

- ✓ Target utilization: 70% (recommended)
- ✓ Minimum capacity: set safe baseline
- ✓ Maximum capacity: set upper limit
- ✓ Scaling cooldown: 60-300 seconds



### SCALING & CAPACITY PLANNING

Plan based on:

- ✓ Peak traffic (reads & writes per second)
- ✓ Item size (affects RCU/WCU usage)
- ✓ Growth projection (daily, weekly, monthly)
- ✓ Global tables (replicated capacity per region)
- ✓ Streams, GSIs, and transactions

#### HEURISTIC FORMULA

$$RCU = \frac{\text{Item Size (KB)} \times (\text{Read Requests/sec})}{4} \quad (\text{Strongly consistent})$$

$$WCU = \frac{\text{Item Size (KB)} \times (\text{Write Requests/sec})}{1}$$



### BEST PRACTICES

- ✓ Design partition keys for even distribution
- ✓ Avoid hot partitions and sequential keys
- ✓ Use sort keys to organize data efficiently
- ✓ Use GSIs to offload access patterns
- ✓ Enable Auto Scaling in provisioned mode
- ✓ Monitor with CloudWatch & Contributor Insights
- ✓ Use On-Demand mode for unpredictable traffic
- ✓ Test at scale before production



### USEFUL COMMANDS (AWS CLI)

```
Describe table
aws dynamodb describe-table \
 --table-name MyTable

List metrics
aws cloudwatch get-metric-statistics \
 --namespace AWS/DynamoDB \
 --metric-name ThrottledRequests --dimensions \
 Name=TableName,Value=MyTable --period 60 --statistics Sum

Contributor Insights (identify hot keys)
aws dynamodb describe-contributor-insights \
 --table-name MyTable

Update capacity
aws dynamodb update-table \
 --table-name MyTable --provisioned-throughput \
 ReadCapacityUnits=100,WriteCapacityUnits=50
```



### KEY TAKEAWAY



Throttling is a signal, not just an error. Good key design, proper capacity planning, and continuous monitoring ensure DynamoDB delivers fast, reliable, and scalable performance for your applications.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

14





# AURORA CLUSTER FAILOVER ISSUES

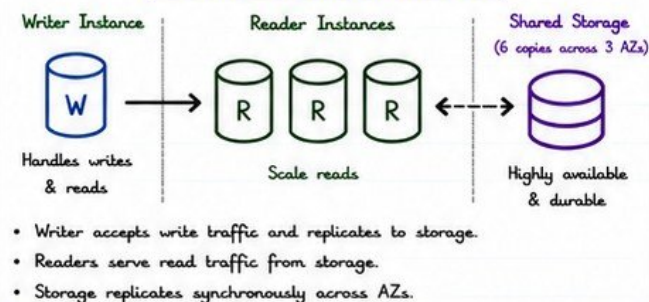


## AT A GLANCE

- ✓ Understand Aurora architecture
- ✓ Analyze failover & writer election
- ✓ Monitor replication & lag
- ✓ Ensure fast recovery
- ✓ Test & validate regularly

Aurora cluster failover issues occur when the primary writer becomes unavailable and the cluster fails to promote a new writer quickly or correctly. This can cause downtime, increased latency, and application errors.

### AURORA ARCHITECTURE OVERVIEW



### FAILOVER FLOW



★ Goal: Minimize downtime and maintain data consistency.

### COMMON SYMPTOMS

- ⚠ Cluster writer endpoint unavailable
- ⚠ High connection errors (5xx, timeouts)
- ⚠ Increased read latency
- ⚠ Replication lag spikes
- ⚠ Writer failover taking too long
- ⚠ Applications unable to write



### WRITER ELECTION (DETAILS)

- 1 Failure detected on writer instance.
- 2 Cluster waits for failure detection timeout (configurable).
- 3 Eligible readers participate in election.
- 4 Quorum reached → new writer elected.
- 5 New writer promoted and endpoints updated.
- 6 Old writer remains down until recovery.

#### TIP

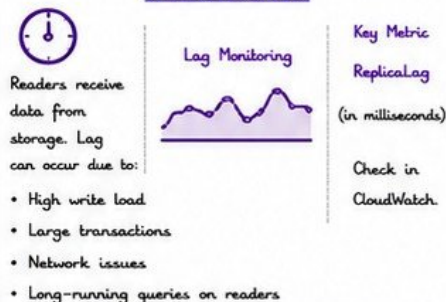
Aurora uses a quorum-based election to ensure data consistency and avoid split brain scenarios.

### ENDPOINTS IN AURORA CLUSTER

- Writer Endpoint**: Always points to the current writer.
- Reader Endpoint**: Load balances across all readers.
- Custom Endpoints**: You define which instances are included.

During failover, endpoints automatically update to the new writer.

### REPLICATION LAG



### INVESTIGATION WORKFLOW

- 1 Check cluster status in RDS Console → Identify writer & reader status
- 2 Check recent events → Look for failover or errors
- 3 Check CloudWatch alarms → CPU, connections, ReplicaLag
- 4 Check ReplicaLag → `rds aurora describe-db-instances`
- 5 Test endpoint connectivity → `nc -zv <endpoint> 3306`
- 6 Review application logs → Look for DB errors/timeouts
- 7 Check failover configuration → Failover timeout, AZs
- 8 If needed, reboot or failover manually → `aws rds reboot-db-instance`



Always correlate metrics, events, and logs.

### RECOVERY OPTIONS

| OPTION              | DESCRIPTION                                 |
|---------------------|---------------------------------------------|
| Automatic Failover  | Always promotes a new writer automatically. |
| Manual Failover     | You trigger failover to a specific reader.  |
| Reboot Writer       | If writer is unresponsive but not failed.   |
| Restore from Backup | If cluster is unrecoverable.                |
| Create New Cluster  | Last resort when data is corrupted or lost. |

#### NOTE

Manual failover will cause a short downtime. Inform your team before triggering.



### BEST PRACTICES

- ✓ Deploy cluster across 3 AZs.
- ✓ Keep Aurora engine version up to date.
- ✓ Use Multi-AZ for high availability.
- ✓ Monitor ReplicaLag and set alarms.
- ✓ Test failover regularly.
- ✓ Use connection pooling & retry logic.
- ✓ Keep transactions short.
- ✓ Backup regularly (automated + manual).



### USEFUL COMMANDS (AWS CLI)

|                                                                                               |                           |
|-----------------------------------------------------------------------------------------------|---------------------------|
| <code>aws rds describe-db-clusters</code>                                                     | Describe cluster status   |
| <code>aws rds describe-db-instances</code>                                                    | List instances in cluster |
| <code>aws rds failover-db-cluster \</code><br><code>--db-cluster-identifier &lt;id&gt;</code> | Trigger manual failover   |
| <code>aws rds reboot-db-instance \</code><br><code>--db-instance-identifier &lt;id&gt;</code> | Reboot an instance        |
| <code>aws rds describe-events \</code><br><code>--source-type db-cluster</code>               | View cluster events       |
| <code>aws cloudwatch get-metric-statistics</code>                                             | Check ReplicaLag metric   |

### KEY TAKEAWAY

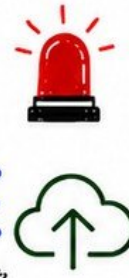
Aurora is designed for high availability, but failover issues can still happen. Monitor health, replication lag, and events continuously. Test failover regularly and have a clear recovery plan.







# LARGE-SCALE AWS MIGRATION FAILURE

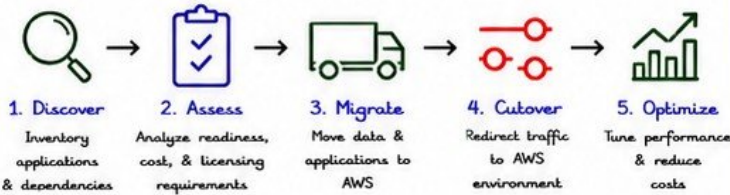


## AT A GLANCE

- ✓ Plan every phase carefully
- ✓ Discover & map everything
- ✓ Test cutover & rollback
- ✓ Monitor closely during migration
- ✓ Have a proven rollback plan

A large-scale migration to AWS fails during cutover, causing extended downtime, data inconsistencies, and performance issues due to poor planning, incomplete testing, and missing rollback procedures.

### MIGRATION LIFECYCLE (AWS MIGRATION HUB)



Use AWS Migration Hub to track progress across tools:  
Application Discovery Service + AWS MGN + DMS + SCT + DataSync

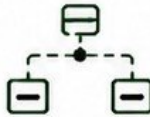
### COMMON FAILURE REASONS

- ✗ Incomplete application discovery
- ✗ Underestimated data transfer time
- ✗ Insufficient performance testing
- ✗ Poor dependency mapping
- ✗ Cutover window too short
- ✗ DNS / routing misconfiguration
- ✗ Rollback plan not tested or missing
- ✗ Stakeholders not aligned
- ✗ Monitoring and alerts not ready



### APPLICATION DISCOVERY (AWS APPLICATION DISCOVERY SERVICE)

- Agentless discovery of servers, apps, & dependencies
- Automatically identifies communication patterns
- Export inventory to Migration Hub
- Validate what to migrate vs. re-architect
- Continuously collect data for accuracy



### CUTOVER PLANNING CHECKLIST

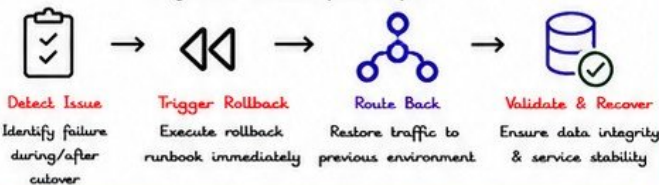
- ☐ Define cutover strategy (big-bang / phased)
- ☐ Create detailed cutover runbook
- ☐ Communicate plan to all stakeholders
- ☐ Pre-cutover validation & smoke tests
- ☐ Freeze changes before cutover
- ☐ Lower TTL for DNS in advance
- ☐ Have a rollback trigger defined
- ☐ Monitor closely during & after cutover

#### CUTOVER TIMELINE EXAMPLE

- T-7d Testing & final validation
- T-1d Freeze & final sync
- T-0h Cutover start
- T+1h Smoke tests & validation
- T+24h Monitor & stabilize

### ROLLBACK STRATEGY

Always have a tested fallback plan!



★ Rollback must be tested in a staging environment before production.

### INVESTIGATION WORKFLOW

- |                               |                                     |
|-------------------------------|-------------------------------------|
| 1 Check CloudWatch alarms     | → Errors, latency, CPU, connections |
| 2 Verify DNS & routing        | → Route53, ALB/NLB, health checks   |
| 3 Validate application health | → Logs, error rates, dependencies   |
| 4 Check data replication      | → DMS / MGN / Database lag          |
| 5 Review cutover steps        | → Runbook vs actual execution       |
| 6 Assess user impact          | → Down services, regions, users     |
| 7 Decide: Recover or Rollback | → Use rollback plan if needed       |
| 8 Communicate & document      | → Status updates & RCA              |

💡 Follow the runbook. Do not improvise during an incident.

### AWS SERVICES USED

- 🔍 AWS Application Discovery Service - Discover & map applications
- 📊 AWS Migration Hub - Centralize & track migration progress
- 🔄 AWS Migration Hub Orchestrator - Automate migration workflows
- 🗄️ AWS Database Migration Service (DMS) - Migrate databases
- 👤 AWS Server Migration Service (MGN) - Replicate servers
- 📁 AWS DataSync - Transfer large datasets
- 🔔 AWS CloudWatch - Monitoring & alerting

### BEST PRACTICES

- ✓ Discover everything before planning
- ✓ Run multiple performance & failover tests
- ✓ Migrate in waves, not all at once
- ✓ Keep systems in sync until final cutover
- ✓ Use Infrastructure as Code (IaC)
- ✓ Automate validation & testing
- ✓ Monitor aggressively during migration
- ✓ Have a tested rollback at all times
- ✓ Document every step & decision



### KEY METRICS TO WATCH

- 📶 Replication Lag  
Seconds behind source
- ⚠️ Application Error Rate  
Errors per second
- 🕒 Latency  
95th percentile response time
- 🕒 Cutover Duration  
Actual vs planned time
- ✅ Rollback Success  
Time to restore service

### KEY TAKEAWAY

Migration is not just technical, it's a program.  
Discover, test, communicate, and always have a rollback.  
A successful migration is a well-rehearsed execution.



### POST-MIGRATION VALIDATION

- ✓ Validate all applications & dependencies
- ✓ Verify data consistency & integrity
- ✓ Monitor performance & capacity
- ✓ Review costs & optimize resources
- ✓ Document lessons learned

### PRO TIP

Treat cutover like a production deployment.  
Rehearse, rehearse, rehearse!  
The best migrations are the boring ones.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA





# ENTERPRISE AWS SECURITY INCIDENT WORKFLOW

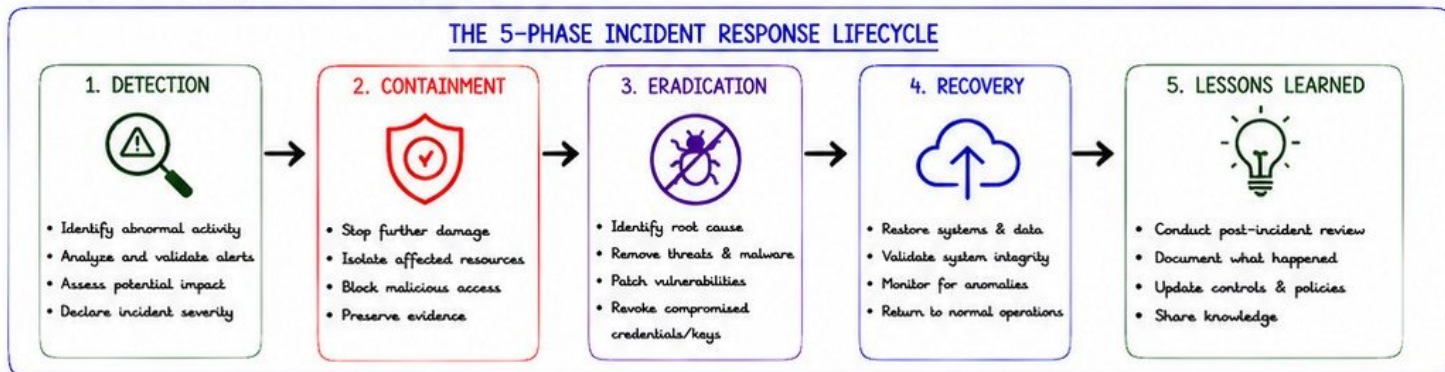


## AT A GLANCE

- ✓ Detect threats early
- ✓ Contain and stop damage
- ✓ Remove the root cause
- ✓ Recover systems safely
- ✓ Learn and improve

A structured incident response process minimizes business impact, protects data, and strengthens your security posture. Follow this proven 5-phase workflow for any AWS security incident.

## THE 5-PHASE INCIDENT RESPONSE LIFECYCLE



### 1. DETECTION

- Sources: GuardDuty, Security Hub, CloudWatch, VPC Flow Logs, WAF, CloudTrail, Config, SIEM.
- Look for anomalies: unusual logins, API calls, data access, network traffic.
- Validate alerts: true positive vs false positive.
- Determine impact: data, users, services, costs.

### INCIDENT SEVERITY MATRIX

| SEVERITY | IMPACT                             | EXAMPLES                                   | RESPONSE TIME |
|----------|------------------------------------|--------------------------------------------|---------------|
| CRITICAL | Severe impact, outage, data breach | Ransomware, data exfiltration, prod outage | < 15 minutes  |
| HIGH     | Major impact, service degraded     | Unauthorized access, privilege escalation  | < 1 hour      |
| MEDIUM   | Moderate impact, limited scope     | Policy violations, misconfigurations       | < 4 hours     |
| LOW      | Low impact, no outage              | Port scans, low risk alerts                | < 24 hours    |

### 3. ERADICATION

- Confirm root cause (logs, timelines, forensics).
- Remove malware, backdoors, unauthorized code.
- Patch vulnerabilities (OS, apps, containers).
- Rotate all secrets: passwords, API keys, certs.
- Review IAM permissions & trust relationships.
- Ensure no persistence mechanisms remain.

### 2. CONTAINMENT

- Isolate compromised instances (Security Groups, NACLs, ACLs).
- Disable or rotate compromised IAM users/keys/roles.
- Block malicious IPs (WAF, NACLs, Route Tables).
- Snapshot volumes and databases (do not modify).
- Enable additional logging if needed.

### INCIDENT RESPONSIBILITY (RACI MODEL)

| TASK                 | SECURITY TEAM | DEVOPS / OPS | APP TEAM | MANAGEMENT |
|----------------------|---------------|--------------|----------|------------|
| Detection & Alerting | R/A           | C            | I        | I          |
| Containment          | R/A           | R            | C        | I          |
| Eradiation           | R/A           | C            | R        | I          |
| Recovery             | C             | R/A          | R        | I          |
| Communication        | R             | C            | C        | R/A        |
| Lessons Learned      | R             | C            | C        | R/A        |

R = Responsible   A = Accountable   C = Consulted   I = Informed

### 5. LESSONS LEARNED

- What happened? (Timeline)
- Why did it happen? (Root cause)
- What worked well?
- What can be improved?
- Update runbooks & playbooks.
- Improve monitoring & alerting.
- Train teams & share learnings.
- Review & update security policies and controls.

### 4. RECOVERY

- Restore from clean backups or rebuild from trusted images.
- Verify data integrity & system functionality.
- Re-enable access gradually.
- Monitor closely (CloudWatch, GuardDuty, SIEM).
- Confirm business services are fully operational.

### COMMUNICATION PLAN

- Internal: Security Team → IT → Management → All Hands
- External (if needed): Customers, Partners, Regulators
- Use a single source of truth (Slack/Teams/War Room)
- Provide regular updates (what we know, actions, ETA)
- Be transparent but avoid sharing sensitive details

### EVIDENCE COLLECTION CHECKLIST

- ✓ CloudTrail logs
- ✓ VPC Flow Logs
- ✓ CloudWatch Logs & Metrics
- ✓ GuardDuty findings
- ✓ API Gateway / ALB logs
- ✓ EBS/RDS snapshots
- ✓ Screenshots & timelines
- ✓ Preserve, don't alter!

### KEY AWS SERVICES FOR INCIDENT RESPONSE

GuardDuty
 Security Hub
 CloudTrail
 CloudWatch
 AWS Config
 IAM
 Systems Manager
 KMS

### GOLDEN RULES

- ✓ Act fast, but stay calm.
- ✓ Contain first, investigate second.
- ✓ Preserve evidence.
- ✓ Communicate early & often.
- ✓ Document everything.
- ✓ Learn and make it better.

Security is a process, not a product.







# ENTERPRISE MULTI-ACCOUNT AWS ARCHITECTURE

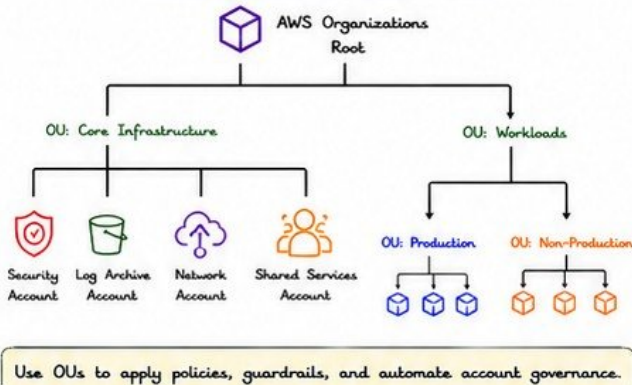


## AT A GLANCE

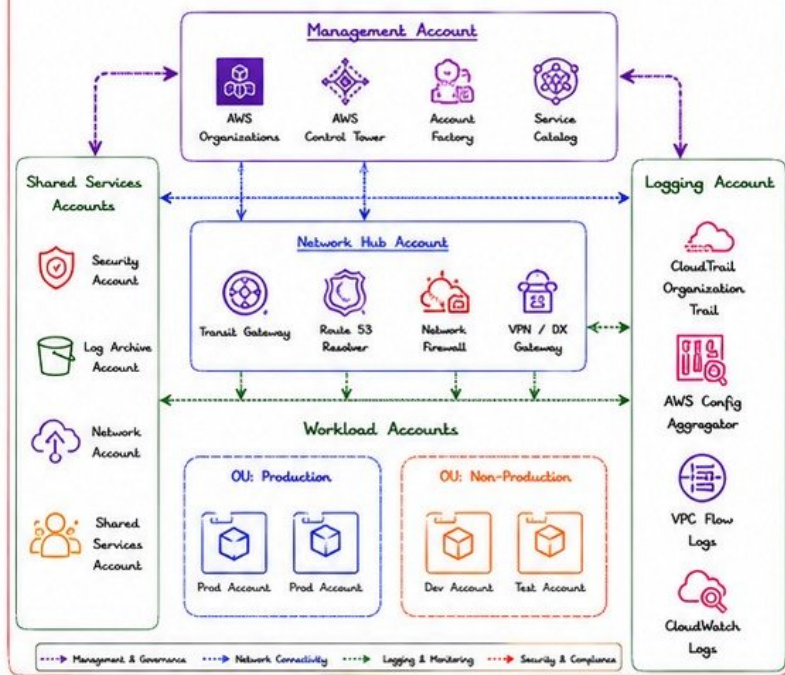
- ✓ Centralized governance
- ✓ Strong security isolation
- ✓ Cost visibility & control
- ✓ Scalable & future-ready
- ✓ Compliance & audit ready

A multi-account architecture using AWS Organizations provides isolation, security, scalability, and governance for enterprise workloads. This is the foundation of a secure AWS Landing Zone.

### 1. AWS ORGANIZATIONS HIERARCHY



### 2. ENTERPRISE MULTI-ACCOUNT ARCHITECTURE



### 3. LANDING ZONE COMPONENTS

- AWS Organizations  
Manage accounts & OUs
- AWS Control Tower  
Automate secure account setup
- Service Control Policies (SCPs)  
Guardrails for security & compliance
- Account Factory  
Automated account provisioning
- Centralized Logging  
Audit, monitor & store logs
- Identity Management  
AWS IAM Identity Center (SSO)
- Networking  
Centralized network & connectivity

### 4. CORE ACCOUNTS PURPOSE

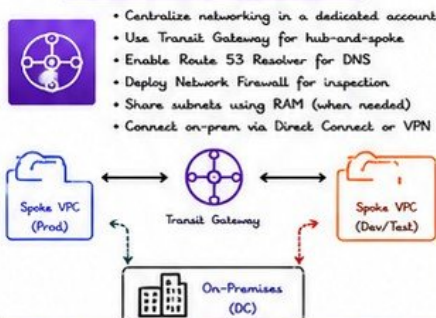
| ACCOUNT TYPE            | PURPOSE                                                                      |
|-------------------------|------------------------------------------------------------------------------|
| Management Account      | Manages AWS Organizations & Control Tower                                    |
| Security Account        | Security tooling, GuardDuty, Security Hub, IAM roles & permission boundaries |
| Log Archive Account     | Centralized log storage (CloudTrail, Config, Flow Logs, CloudWatch Logs)     |
| Network Account         | Transit Gateway, Route 53 Resolver, Network Firewall, VPN, DX                |
| Shared Services Account | Shared resources (AD, Bastion, CI/CD, Artifacts, License Mgmt)               |
| Workload Accounts       | Host application workloads (Prod/Non-Prod)                                   |

### 5. SECURITY & GOVERNANCE

- ✓ Apply SCPs to enforce guardrails
- ✓ Use least privilege IAM & permission boundaries
- ✓ Centralize CloudTrail with Organization Trail
- ✓ Enable AWS Config conformance packs
- ✓ Encrypt everything (KMS, S3, EBS, RDS)
- ✓ Use AWS IAM Identity Center (SSO)
- ✓ Automate compliance & patch management
- ✓ Monitor with GuardDuty, Security Hub & CloudWatch



### 6. NETWORK HUB (BEST PRACTICES)



### 7. ENTERPRISE BEST PRACTICES

- ✓ Design for isolation & blast radius reduction
- ✓ Automate account provisioning & baseline setup
- ✓ Tag everything (cost, owner, env, app)
- ✓ Centralize logging, monitoring & security
- ✓ Implement backup & disaster recovery
- ✓ Regularly review access & permissions
- ✓ Use IaC (Terraform/CloudFormation) for consistency
- ✓ Document architecture & runbooks



### KEY TAKEAWAY

A well-designed multi-account architecture is the backbone of enterprise AWS. It improves security, governance, scalability, and operational efficiency while reducing risk and cost. Build it right, and your cloud foundation will scale with your business.







# PRINCIPAL / STAFF LEVEL AWS SYSTEM DESIGN INTERVIEW SCENARIOS



## AT A GLANCE

- ✓ Design for scale and failure
- ✓ Discuss trade-offs clearly
- ✓ Consider security by default
- ✓ Think cost vs performance
- ✓ Drive for operational excellence
- ✓ Align solution with business goals

Example scenarios and discussion points for Principal and Staff level AWS System Design interviews. Think big, trade-offs, ownership, and business impact.

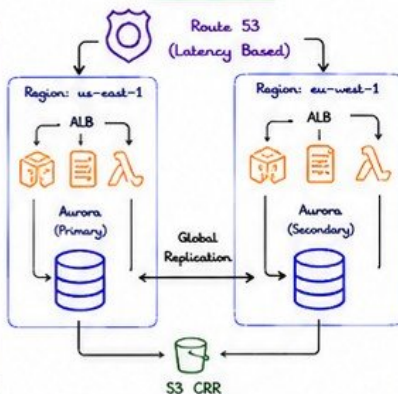
### 1. GLOBAL MULTI-REGION ARCHITECTURE

SCENARIO: Design a globally available application with < 1s latency worldwide and zero single points of failure.

#### DESIGN PATTERN

##### KEY COMPONENTS

- Route 53 (Latency based)
- CloudFront (Global CDN)
- AWS Global Accelerator
- Multi-Region (Active-Active)
- DynamoDB Global Tables
- S3 Cross-Region Replication
- Aurora Global Database



#### DISCUSSION POINTS

Failover strategy, data consistency, DNS failover, RPO/RTO, compliance, cost.

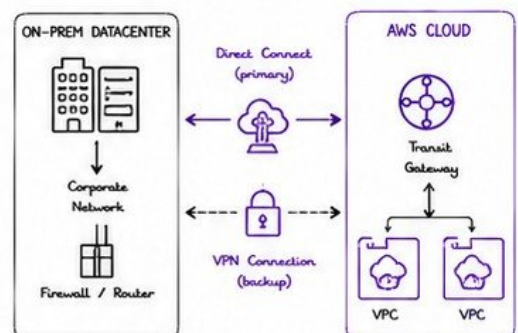
### 2. HYBRID CLOUD ARCHITECTURE

SCENARIO: Design a hybrid architecture connecting on-premises datacenter to AWS securely and reliably.

#### KEY COMPONENTS

- Direct Connect (primary)
- Site-to-Site VPN (backup)
- Transit Gateway
- VPC Segmentation
- On-prem AD Integration
- Route 53 Resolver
- AWS Outposts (optional)

#### ARCHITECTURE OVERVIEW



#### DISCUSSION POINTS

High availability, bandwidth planning, security, routing, cost, OPEX vs CAPEX.

### 3. SECURITY TRADE-OFFS

SCENARIO: A startup needs to launch fast but also meet strong security requirements.

#### KEY TRADE-OFFS

| SECURITY              | VELOCITY / USABILITY |
|-----------------------|----------------------|
| Least privilege IAM   | Dev productivity     |
| Network isolation     | Simple architecture  |
| Encryption everywhere | Performance impact   |
| MFA / SSO             | User friction        |
| Centralized logging   | Cost & complexity    |

#### DISCUSSION POINTS

Risk appetite, layered security, guardrails, enablement.

### 4. COST VS PERFORMANCE

SCENARIO: Design an API service with millions of requests. Optimize for cost vs performance.

#### COST OPTIMIZED

- API Gateway (HTTP)
- Lambda + DynamoDB
- S3 + CloudFront
- Savings Plans
- Right-size resources
- Async processing (SQS, EventBridge)

#### PERFORMANCE OPTIMIZED

- ALB + ECS / EKS
- Provisioned RDS
- ElastiCache (Redis)
- Global Accelerator
- Multi-AZ / Multi-Region
- Provisioned capacity



#### DISCUSSION POINTS

Traffic patterns, latency, concurrency, scaling, cost controls.

### 5. RESILIENCY DISCUSSIONS

SCENARIO: Design for extreme availability with zero downtime deployments.

#### KEY STRATEGIES

- Multi-AZ by default
- Auto Scaling & health checks
- Blue/Green or Canary deployments
- Stateless services
- Backups & point-in-time recovery
- Chaos testing & game days
- DR: Warm Standby / Multi-Region



#### DISCUSSION POINTS

SLA/SLO, RTO/RPO, fault isolation, testing.

### WHAT SENIOR INTERVIEWERS EVALUATE

|                           |                                                               |
|---------------------------|---------------------------------------------------------------|
| Business Understanding    | Do you ask the right questions and understand trade-offs?     |
| Architecture Depth        | Do you know AWS services deeply and how they integrate?       |
| Trade-off Analysis        | Do you clearly explain pros/cons and justify decisions?       |
| Scalability & Reliability | Can your design handle massive scale and failures?            |
| Security Mindset          | Is security built-in, not bolted-on?                          |
| Cost Awareness            | Do you design with cost efficiency in mind?                   |
| Ownership & Leadership    | Can you drive alignment, think long-term, and take ownership? |

### DESIGN FRAMEWORK (USE EVERY TIME)

1. Clarify Requirements  
Ask questions, define scope, constraints, assumptions.
2. High Level Design  
Sketch the architecture & major components.
3. Deep Dive  
Pick components, discuss scale, data flow, failure scenarios.
4. Trade-offs  
Compare options, justify choices with data.
5. Security  
Discuss identity, network, data protection, compliance.
6. Operations  
Monitoring, logging, alerting, CI/CD, runbooks.
7. Cost Optimization  
Right-sizing, savings plans, lifecycle policies.
8. Wrap Up  
Summarize, risks, future improvements.

### SENIOR DESIGN PRINCIPLES

- ✓ Stop guessing, start measuring
- ✓ Design for failure, not for success
- ✓ Automate everything
- ✓ Make security invisible
- ✓ Prefer managed services
- ✓ Simplicity before sophistication
- ✓ Build for people, not just systems

“Great architects design systems that are simple, secure, scalable, cost-efficient, and a joy to operate.”



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA





# ENTERPRISE AWS OPERATIONS PLAYBOOK



## AT A GLANCE

- ✓ Run a reliable, secure and efficient AWS operation.
- ✓ Review regularly.
- ✓ Automate everything.
- ✓ Improve continuously.

A proven operational cadence keeps your AWS environment secure, reliable, cost-efficient, and ready for scale. Follow this playbook and never miss a critical review.

### 1. DAILY OPERATIONAL CHECKLIST



- ☐ Check AWS Health Dashboard
- ☐ Review CloudWatch alarms
- ☐ Review high priority alerts
- ☐ Check critical system dashboards
- ☐ Verify backup & snapshot status
- ☐ Check failed jobs (Lambda, Batch, Glue)
- ☐ Review security findings (GuardDuty, Security Hub)
- ☐ Check auto scaling activities
- ☐ Validate daily log ingestion (CloudTrail, VPC Flow, ALB)
- ☐ Monitor application performance (Latency, Errors, CPU, Memory)
- ☐ Check cost anomalies / daily spend
- ☐ Confirm operational tickets & on-call handover

#### KEY COMMANDS

- `aws health describe-events`
- `aws cloudwatch get-metric-statistics`
- `aws cloudwatch describe-alarms`
- `aws ce get-cost-and-usage --time-period Start=...End=...`



### 2. WEEKLY ARCHITECTURE REVIEW



- ☐ Review architecture diagrams
- ☐ Verify service limits & quotas
- ☐ Check auto scaling & capacity trends
- ☐ Review high availability & failover tests
- ☐ Review network configuration & routing
- ☐ Check for Single Points of Failure (SPOF)
- ☐ Review dependency mapping
- ☐ Validate tagging compliance
- ☐ Review cost optimization opportunities
- ☐ Document architecture changes

#### FOCUS AREAS

- Resiliency
- Scalability
- Performance
- Cost efficiency
- Simplicity
- Operational excellence



### 3. MONTHLY SECURITY REVIEW



- ☐ Review Security Hub findings & compliance score
- ☐ Review GuardDuty alerts & threat intelligence
- ☐ Review IAM users, roles & permission changes
- ☐ Check Access Analyzer findings
- ☐ Review encryption (KMS usage & key rotation)
- ☐ Review security group & NACL rules
- ☐ Review WAF rules & blocked requests
- ☐ Review vulnerability scans (Inspector, ECR)
- ☐ Review Secrets Manager rotation
- ☐ Review incident tickets & security trends

#### SECURITY PRINCIPLES

- Least privilege
- Encrypt everything
- Detect continuously
- Respond quickly
- Review regularly



### 4. QUARTERLY DISASTER RECOVERY DRILLS



- ☐ Test backups restoration
- ☐ Test cross-region failover
- ☐ Validate RPO & RTO targets
- ☐ Test database PITR recovery
- ☐ Test DNS failover (Route 53)
- ☐ Test application recovery
- ☐ Validate automation runbooks
- ☐ Review DR documentation
- ☐ Collect lessons learned
- ☐ Update DR plan

#### DR CHECKLIST

- Backups
- Failover
- Recovery
- Automation
- Communication
- Documentation



### 5. ANNUAL WELL-ARCHITECTED REVIEW



#### THE 6 PILLARS

- Operational Excellence
- Security
- Reliability
- Performance Efficiency
- Cost Optimization
- Sustainability

#### REVIEW ACTIVITIES

- ☐ Evaluate current state for each pillar
- ☐ Identify improvement opportunities
- ☐ Prioritize & build roadmap
- ☐ Measure improvements
- ☐ Update documentation & runbooks
- ☐ Align with business objectives

### 6. ENTERPRISE OPERATIONS BEST PRACTICES

- ☐ Automate everything possible (IaC + CI/CD)
- ☐ Infrastructure as Code for consistency
- ☐ Monitor everything (Logs, Metrics, Traces)
- ☐ Centralize logging (CloudWatch, S3, OpenSearch)
- ☐ Use standardized tagging strategy
- ☐ Apply least privilege access everywhere
- ☐ Track costs daily and optimize
- ☐ Document everything (Runbooks, SOPs, Diagrams)
- ☐ Conduct regular training & knowledge sharing
- ☐ Continuously improve and iterate

#### AUTOMATION FIRST

Automate deployments, monitoring, alerts, remediation, and reporting. Reduce manual work and human errors.



## FINAL VERIQTA ENTERPRISE ENGINEERING PRINCIPLES



#### SECURITY BY DEFAULT

Security is not an afterthought. Build it in.



#### RELIABILITY FIRST

Design for failure. Build for resiliency.



#### AUTOMATE & SCALE

Use automation and scalable architectures.



#### DATA-DRIVEN DECISIONS

Measure, analyze, and make smart decisions.



#### OWNERSHIP & ACCOUNTABILITY

Own your systems. Own the outcomes.



#### CONTINUOUS IMPROVEMENT

Review, learn, improve, and repeat.

Great operations is not about avoiding incidents. It's about preparing, responding, and evolving.

VERIQTA



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA